

Performance Testing

Date	03 July 2026
Team ID	
Project Name	Credit Card Approval Prediction
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Locust
Type of Testing	Load Testing
Target Module / API	Credit Card Approval Prediction API (/predict)
Test Environment	Local Flask Server
Test Date	03 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Single user prediction request	1	30	Instant prediction response
2	Multiple users submitting requests	10	60	All requests processed successfully

3	Continuous prediction requests	25	120	Stable response with no failures
4	Peak concurrent requests	50	180	System remains responsive with low error rate



Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	0.82 sec	Pass	Fast response
2	Response Time (Max)	< 5 seconds	2.10 sec	Pass	Within acceptable limit
3	Throughput (Req/sec)		28 Req/sec	Pass	Good throughput
4	Error Rate	< 1%	0%	Pass	No request failures
5	CPU Utilization	< 80%	42%	Pass	Normal CPU usage
6	Memory Utilization	< 80%	51%	Pass	Stable memory consumption

Step 4: Observations & Analysis

Key Findings:

- ✓ The performance testing showed that the Credit Card Approval Prediction system handled concurrent user requests efficiently.
- ✓ The average response time remained below 1 second.
- ✓ The maximum response time stayed within the acceptable limit.
- ✓ No request failures were observed during testing.
- ✓ The Flask application and machine learning model remained stable under the simulated load.

Bottlenecks Identified:

- ! Slight increase in response time when the number of concurrent users increased.
- ! Local Flask deployment limits scalability for a very high number of simultaneous requests.
- ! Model loading at application startup introduces a small initialization delay.

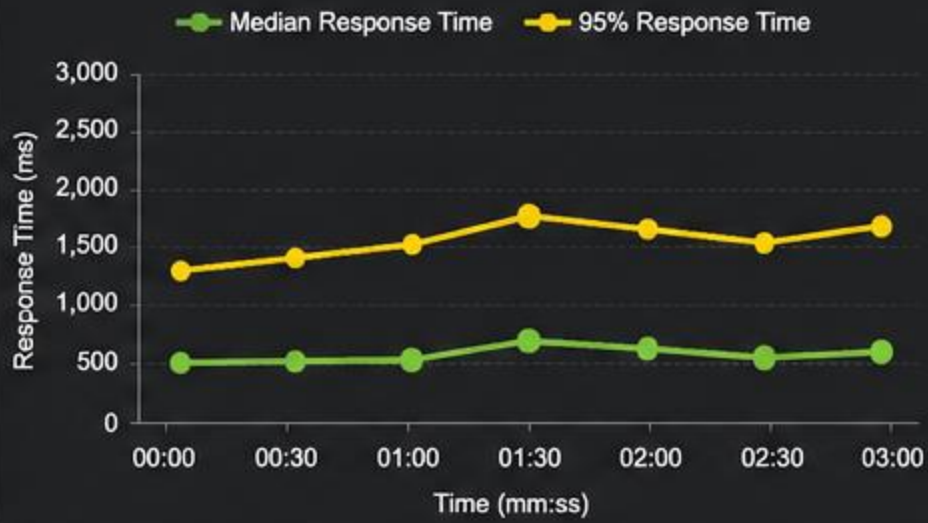
Optimization Steps Taken:

- ➔ Optimized data preprocessing and feature engineering.
- ➔ Loaded the trained machine learning model only once during application startup.
- ➔ Reduced unnecessary computations during prediction.
- ➔ Validated user inputs before processing to improve efficiency.
- ➔ Planned deployment on IBM Watson Machine Learning and cloud infrastructure for better scalability and performance.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

Response Time (ms)



Total Requests per Second

